

UXL SKILLS EVALUATOR

oneMath Skill

Domains, dispatch, queues, backends, and linking

September 2026 • 3 of 6 evaluation tasks implemented

Why this is useful to oneMath at first glance

- **Stop naming confusion** — The skill distinguishes open-source oneMath from Intel oneMKL and asks agents to name the actual backend.
- **Choose the right dispatch path** — Runtime dispatch, compile-time selectors, and RNG device API have different build and dependency contracts.
- **Make queue ownership explicit** — Agents preserve caller-owned execution and event dependencies instead of creating hidden queues.
- **Prevent compatibility folklore** — Backend/domain/OS/hardware claims require current support-matrix verification.

What the skill tells an agent today

- **Classify the domain** — BLAS, LAPACK, RNG, DFT, or sparse BLAS before choosing an API path.
- **Select a usage model** — Runtime dispatch for one link/load path; compile-time dispatch for explicit/static backend control; device API for supported in-kernel RNG.
- **Write the math contract** — Precision, layout, dimensions, leading dimensions, batch strides, and event dependencies.
- **Diagnose build and runtime separately** — Compiler, wrapper libraries, package discovery, runtime loader paths, and device visibility.
- **Test small known answers first** — Then measure representative sizes with dispatch and transfer scope stated.

Evaluation inventory: 3 implemented, 3 targeted gaps

- **Implemented** — Missing runtime backend library; packed-band storage fixtures; deprecated header include.
- **Incident evidence** — Two implemented tasks are grounded in upstream maintainer incidents.
- **Planned device work** — RNG device event chain and third-party backend wrapper on a declared target.
- **Planned toolchain work** — Dispatch-overhead benchmark with a pinned hosted toolchain.
- **What this proves** — Current tasks cover integration and API-contract reasoning, not the full backend matrix.

How we chose the content

- **Official project material** — Usage-model docs, build/link instructions, backend integration guidance, examples, releases, and support matrices.
- **Common boundary failures** — Wrong dispatch assumption, missing runtime libraries, storage layout mistakes, include/package drift.
- **Portable wording** — The skill names multiple backend families while requiring verification for the exact domain and version.
- **Visible gaps** — Device events, third-party wrappers, and dispatch measurement stay planned until their environments are credible.

What project ownership would look like

- **Review the decision tree** — Domain, host/device API, runtime versus compile-time dispatch, compiler, and backend.
- **Curate current sources** — Keep support-matrix and release-specific claims fresh; delete stale compatibility advice.
- **Shape the evaluation backlog** — Replace constructed tasks with authentic integration failures when possible.
- **UXL keeps the machinery** — Common validators, Harbor, dashboards, hosted toolchains, and runner contracts remain shared.

A focused 30-minute maintainer review

- **1. Confirm terminology** — oneMath versus oneMKL; current API names; supported compatibility includes.
- **2. Check the dispatch guidance** — Is the runtime/compile-time/device decision tree accurate and complete enough?
- **3. Prioritize the three gaps** — Which one best represents real maintainer pain and can be reproduced?
- **4. Name an owner** — One reviewer for source and support-matrix freshness.
- **Outcome** — Agents arrive at oneMath PRs with the domain, usage model, backend, link contract, and evidence already stated.